DP REVISION NOTEBOOK

LIS & Subsequence Dynamic Programming

From recognizing the pattern to deriving the state, transition, and implementation

Your central lesson

A hard-looking subsequence problem often becomes manageable when you identify the shape of the sequence, decide what information must be remembered, and connect the problem to a pattern you already know.

Prepared from your recent LIS / subsequence practice and the LeetCode problems visible in your progress screenshot.

Use this as a thinking notebook, not a collection of solutions to memorize.

1. Your current learning map

The screenshot shows a useful progression: classic LIS, variations where the relation between neighbouring/selected values changes, problems with extra state, and problems that combine two subsequence directions.

Problem	Main pattern / observation	What it trains
300. Longest Increasing Subsequence	A subsequence is built while tracking the previous chosen value/index.	The foundation: state, transition, $O(n^2)$ and $O(n \log n)$ ideas.
368. Largest Divisible Subset	Sort first; divisibility gives a predecessor relation.	Transform a subset problem into LIS-like DP + reconstruction.
646. Maximum Length of Pair Chain	Sort pairs; choose compatible next pairs.	Recognize both DP and greedy formulations.
1048. Longest String Chain	Sort by word length; one word must be formed by adding one character.	Define a custom predecessor relationship.
1671. Minimum Number of Removals to Make Mountain Array	Increasing side + decreasing side meet at a valid peak.	Combine left-to-right and right-to-left DP.
2370. Longest Ideal Subsequence	The previous character must be within a value-distance constraint.	Replace previous-index scanning with a small character-state DP.
3201. Find the Maximum Length of Valid Subsequence I	The validity rule depends on parity / remainder relationships.	Track a compact remainder or category state.
1420. Build Array Where You Can Find The Maximum Search Cost	Index, search cost, and maximum-so-far all affect the future.	Multidimensional DP and careful state bounds.
2926. Maximum Balanced Subsequence Sum	A transformed compatibility condition governs which earlier element can extend the subsequence.	Advanced subsequence DP; optimize predecessor queries.
1911. Maximum Alternating Subsequence Sum	The contribution alternates by position in the chosen subsequence.	DP over a small number of states.
198. House Robber	Take or skip; adjacent choices conflict.	The basic choice-based DP pattern.

Note on your attempts: 2370 showed a time-limit issue and 2926 showed a memory-limit issue in the screenshot. Treat these as useful feedback: the recurrence may be valid while the implementation's complexity or storage is not.

A good classification habit

For every new problem, ask: (1) Is it a subsequence? (2) What makes one chosen element compatible with another? (3) What information about the past changes my future choices? (4) Can I sort or transform the values to simplify compatibility? (5) Can I optimize the predecessor search?

2. The foundation: Longest Increasing Subsequence

A subsequence keeps the relative order of selected elements, but elements do not need to be adjacent. For example, in [3, 1, 5, 2, 6], [1, 2, 6] is a subsequence.

The kev observation

At index i, if we want an increasing subsequence ending at nums[i], the previous selected element must be at some index j where j < i and nums[j] < nums[i]. So every valid j is a possible predecessor.

O(n²) DP definition

dp[i] = length of the longest strictly increasing subsequence that ends exactly at index i.

Transition:

dp[i] = 1 + max(dp[j]) over all j < i with nums[j] < nums[i]. If no such j exists, dp[i] = 1. The final answer is max(dp[i]).

Why this state works

We only need the best increasing subsequence ending at j to extend it with nums[i]. We do not need the entire subsequence during the length calculation. The state stores exactly the information needed by future positions.

C++ template — O(n²)

```
int lengthOfLIS(vector<int>& nums) {
    int n = nums.size();
    vector<int> dp(n, 1);

    int ans = 1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (nums[j] < nums[i]) {
                dp[i] = max(dp[i], dp[j] + 1);
            }
        }
        ans = max(ans, dp[i]);
    }
    return ans;
}
```

The alternative top-down state you practised

solve(idx, prevIndex) means: from idx onward, what is the best length I can build if the last selected index is prevIndex? The choices are skip idx, or take idx if prevIndex == -1 or nums[idx] > nums[prevIndex]. This state is natural for recursion, but it has O(n²) states and O(n) work/branching per state in a straightforward implementation.

The -1 indexing trick

If you store prevIndex in an array, use prevIndex + 1. Then prevIndex = -1 maps to column 0, prevIndex = 0 maps to column 1, and so on. This avoids a negative array index.

O(n log n) recognition

The optimized LIS method maintains a tails array: tails[len-1] is the smallest possible ending value of an increasing subsequence of length len. For each x, use lower_bound to find the first element >= x and replace it; if none exists, append x. The size of tails is the LIS length. Important: tails is not necessarily an actual subsequence.

3. The subsequence pattern library

Pattern A — Increasing / decreasing subsequence

Look for a sequence where each selected value must be greater than (or smaller than) the previous selected value. Start with LIS-style predecessor DP. If the direction is reversed, think about reversing the traversal or defining a decreasing state.

Pattern B — Bitonic / mountain subsequence (LeetCode 1671)

The mountain has two phases: strictly increasing up to a peak, then strictly decreasing after it. A peak at index i must have a non-empty increasing side and a non-empty decreasing side.

Define inc[i] as the LIS length ending at i, and dec[i] as the longest strictly decreasing subsequence starting at i. The second state is computed from right to left. If both inc[i] > 1 and dec[i] > 1, i is a valid peak. Mountain length at i = inc[i] + dec[i] - 1. Answer = n - maximum valid mountain length.

The observation that unlocked 1671

Do not try to build a brand-new 3D state for increase/decrease immediately. The problem already separates naturally into two known subproblems. Compute the best rising arm and the best falling arm independently, then join them at the same peak. The peak is counted twice, so subtract one.

Pattern C — Sort, then do LIS-like DP

Some problems do not look like LIS because their elements are pairs, strings, or sets. Sorting can establish a useful order, after which you define whether one item can precede another. Examples: 368 (divisibility), 646 (pair chain), 1048 (string chain). Always verify that the sort order preserves the problem's valid ordering.

Pattern D — Subsequence with a custom predecessor rule

The core question is still: which earlier selected item can extend the current item? The rule may be nums[j] < nums[i], difference within k, divisibility, one-character insertion, or a transformed inequality. Write this compatibility rule explicitly before writing the DP.

Pattern E — Small extra state (parity / remainder / alternating)

If future validity depends on the previous selected element's category rather than its exact index, track that category. For parity-based problems, the category may be 0/1. For remainder problems, it may be modulo m. For alternating sums, it may be whether the next chosen value contributes positively or negatively.

Pattern F — Multiple state variables

Use a multidimensional state only when each dimension represents information that changes future decisions. For 1420, the index, current search cost, and maximum value so far all matter. Before adding a dimension, ask whether it can be derived from another state or compressed.

Pattern G — Optimize the predecessor search

A valid $O(n^2)$ recurrence may still be too slow. If each state scans all previous indices, look for sorted keys, binary search, Fenwick tree, segment tree, or a bounded alphabet. 2370 is a good reminder that scanning every prior index can be unnecessary when the value domain is small.

4. Your solved / attempted problems — revision cards

300. Longest Increasing Subsequence

Recall: dp[i] is the best increasing subsequence ending at i. Try both the take/skip recursion with (idx, prevIndex) and the O(n^2) ending-at-i formulation. Know why the optimized tails + binary search method works and why tails is not the actual chosen subsequence.

368. Largest Divisible Subset

Observation: sort the numbers. For positive values, if nums[j] divides nums[i] and j < i, nums[j] can precede nums[i] in a divisible chain. This is LIS-like DP with divisibility replacing the less-than comparison. Store parent/previous indices if you need to reconstruct the subset, not just its length.

646. Maximum Length of Pair Chain

Observation: one pair can follow another when the previous pair's end is smaller than the next pair's start. Sorting by end makes a greedy strategy possible; a DP formulation can also consider all compatible previous pairs. Learn why greedy is sufficient after sorting by end.

1048. Longest String Chain

Observation: sort words by increasing length. A predecessor is exactly one character shorter, and deleting one character from the longer word should produce the shorter word. DP for a word is 1 + the best chain among its valid predecessors. A hash map can store the best chain length for words already processed.

1671. Minimum Number of Removals to Make Mountain Array

Observation: find the longest valid mountain subsequence, not the removals directly. Compute LIS ending at each index and decreasing subsequence starting at each index. Exclude indices where either side has length 1. Minimum removals = n - longest mountain length.

2370. Longest Ideal Subsequence

Observation: the previous character only matters through its alphabet position and the allowed distance k. A DP over the 26 lowercase letters can avoid scanning all earlier indices. For a character c, inspect allowable predecessor letters in [c-k, c+k], then update the best chain ending in c. Your TLE is a complexity signal: identify the small value domain and exploit it.

3201. Find the Maximum Length of Valid Subsequence I

Observation: the validity condition is based on parity/remainder behaviour, so the exact previous index is not the important state. Track the best subsequence lengths for the relevant parity categories / possible alternating pattern. This is an example of compressing previous-element information into a tiny state.

1420. Build Array Where You Can Find The Maximum Search Cost

Observation: the state needs index, search cost, and maximum so far because each affects what future choices do. Your practiced state was solve(idx, searchCost, maxSoFar). For bottom-up conversion, reason from the transition dependencies rather than blindly reversing every loop. Bound dimensions from constraints and initialize unreachable states carefully.

2926. Maximum Balanced Subsequence Sum

Observation: this is an advanced weighted subsequence problem. Transform the condition so compatibility can be queried using a key derived from each index/value, then maintain the best previous sum among compatible keys. A straightforward scan over all earlier indices can be too slow; a full DP table can also waste memory. Your MLE is a reminder to inspect state count and whether a data structure can replace a large table.

1911. Maximum Alternating Subsequence Sum

Observation: choosing an element changes whether the next chosen element is added or subtracted. You only need a small number of states representing the best result when the next contribution is positive or negative (or even/odd position in the chosen subsequence).

198. House Robber

Observation: at each position, either take the current value and skip the adjacent one, or skip the current value. This is the classic take/skip DP and a foundation for recognizing choice conflicts.

5. A repeatable method for solving new DP problems

Step 1 — Translate the goal into a mathematical object

Is the answer a longest/shortest subsequence, a maximum sum, a count of ways, or a minimum number of removals? If the prompt asks for removals, ask whether it is equivalent to keeping the largest valid subsequence: removals = n - kept.

Step 2 — Write the validity rule

For a selected sequence a1, a2, ..., write the exact condition between consecutive selected elements. Examples: a(j) < a(i); end(pair j) < start(pair i); word j is one character shorter; parity relation holds. This rule often reveals the predecessor loop.

Step 3 — Decide what the state must remember

Ask: if I pause at this point, what facts about the past are needed to make every future decision? Common choices are current index + previous selected index; current index with best answer ending here; current character/parity/remainder; or index + a few global quantities.

Step 4 — Derive choices and transition

For take/skip problems, list both choices. For ending-at-i subsequence DP, enumerate all valid predecessors j. State the base case in plain language before coding. Then write the recurrence.

Step 5 — Look for structure before adding dimensions

Can sorting establish order? Can the problem be split into left and right halves? Can a category replace the exact previous index? Is the value range small? Can a maximum over earlier states be queried with binary search or a tree? These questions often turn a correct but slow DP into an accepted one.

Step 6 — Calculate complexity before implementation

Approach	Typical complexity	When it is reasonable
Two loops over indices	$O(n^2)$ time, $O(n)$ memory	Moderate n; simple predecessor DP.
Index + previous-index memo	$O(n^2)$ states; often $O(n^2)$ or $O(n^3)$ work depending on transitions	When take/skip state is natural and constraints allow.
Binary-search LIS tails	O(n log n) time, O(n) memory	Strict LIS length; reconstruction needs extra bookkeeping.
Small category DP	O(n × categories)	Parity, alphabet, remainder, small bounded state.
Fenwick / segment tree	Usually O(n log n)	Need best value over a prefix/range of transformed keys.
3D DP	Product of all dimension sizes	Only when all dimensions are genuinely necessary and constraints permit.

Step 7 — Test edge cases

Try n = 1; strictly increasing; strictly decreasing; all equal; duplicates around a possible peak; no valid mountain; peak at an endpoint; smallest and largest allowed values. For strict mountain problems, equal adjacent values cannot be part of either strict arm.

Your 20-minute independent attempt routine

Time	What to do
0–5 min	Restate the goal; identify subsequence / subarray / substring; inspect constraints.
5–10 min	Write brute-force choices and a recursive state in words.

Time	What to do
10–15 min	Write the compatibility rule; compare with known patterns; consider sorting or splitting directions.
15–20 min	Derive recurrence, base cases, and complexity. Only then code.
After 20 min	If stuck, take one small hint. Do not jump straight to full code. After solving, write the unlocking observation.

6. Mistakes to watch for

• **Confusing subsequence with subarray:** subsequence elements need not be adjacent; subarray elements must be contiguous.

• **Using the wrong LIS meaning:** LIS ending at i and LIS starting at i are different states. Be explicit about direction.

• **Forgetting strictness:** nums[j] < nums[i] is different from nums[j] <= nums[i]. Match the problem statement.

• **Combining two sides incorrectly:** when both sides include a shared peak, subtract one. Also ensure both arms contain at least two elements.

• **Assuming sorting is always safe:** sorting changes order. Prove that the problem permits this transformation.

• **Adding a 3D state too early:** each dimension multiplies memory. First check if a category or two separate 1D DPs are enough.

• **Ignoring constraints:** an $O(n^2)$ recurrence may be correct but too slow. A large 3D table may cause MLE.

• **Thinking tails is the actual LIS:** in the binary-search method, tails stores best possible tail values, not necessarily one valid subsequence.

• **Not reconstructing when required:** a length DP alone does not give the chosen elements. Store parent indices or predecessor information.

7. Quick revision sheet

If the prompt says…	Think…	First question to ask
Longest increasing/decreasing	LIS-style DP	What can be the previous selected element?
Increasing then decreasing	Bitonic / mountain	Can I compute the two arms independently?
Minimum removals to satisfy a sequence property	n – longest valid kept subsequence	What is the longest sequence I can keep?
Pairs / words / divisible values	Sort + custom predecessor DP	What makes item j a predecessor of item i?
Parity / modulo / alternating	Small category-state DP	Can I store the previous category instead of its index?
Extra global conditions	Multidimensional DP	Which quantities truly affect future choices?
Correct DP but TLE	Optimize transitions	Can I use sorting, binary search, bounded states, Fenwick/segment tree?
MLE	Compress state / memory	Do I need every dimension and every state stored?

Before you submit — DP checklist

■ State has a precise sentence meaning.
■ Transition follows directly from the problem's validity rule.
■ Base cases match the state meaning.
■ Every state dependency is computed / memoized correctly.
■ Strict vs non-strict comparisons are correct.
■ Complexity fits the constraints.
■ Integer type is large enough (use long long when sums may overflow).
■ Edge cases have been tested.

8. Active recall: questions to answer without looking

Try to answer these aloud or in writing. If you cannot answer one, revisit only that section and then close the notebook again.

#	Recall question
1	What exactly does dp[i] mean in the $O(n^2)$ LIS formulation?
2	Why must the predecessor index j be smaller than i?
3	Why does nums[j] < nums[i] appear in the LIS transition?
4	What is the difference between LIS ending at i and LIS starting at i?
5	Why is 1671 solved by combining two directional DPs?
6	Why do we subtract one when joining the increasing and decreasing arms?
7	Why must a valid mountain peak have both arm lengths greater than one?
8	How does sorting help in 368, 646, and 1048? What is the predecessor rule in each?
9	When can previous-index state be compressed to parity, remainder, or character?
10	What does a TLE tell you about the transition, and what does an MLE tell you about the state storage?
11	When would you consider binary search, a Fenwick tree, or a segment tree?
12	Why is the tails array in O(n log n) LIS not necessarily a real subsequence?

Final reminder

The goal is not to recognize every problem instantly. The goal is to build a reliable chain of reasoning: understand the required sequence → write the compatibility rule → define the smallest useful state → derive the transition → check complexity → implement. Repeat that process until the patterns become familiar.

Your next practice rule: solve first, ask for a hint second, read the full solution last. After every problem, record the one observation that unlocked it.